NETWORK PROGRAMMING AND CLIENT SERVER PROGRAMMING

Assignment: Simple HTTP Client using Socket Programming in C

Field	Details
Student Name	UJJWAL JAIN
Student ID	2024EB02528
Source file	`http_client.c`
Platform	C on Linux, POSIX socket API

1. INTRODUCTION

HTTP works on the client-server model over TCP. The client opens a TCP connection to the server on port 80, sends a request message and the server replies with a status line, headers and the requested object. In this assignment a small HTTP client is written in C. It takes the server as a hostname or an IP address on the command line, sends a GET request and prints the full reply from the server.

2. DESIGN OF THE PROGRAM

Order in which the functions are called:

```
main() -> interact_with_server() -> create_socket_and_connect()
       -> send_http_get_request() -> receive_http_response() -> close()
```

Steps followed by the program:

1. `main()` checks the arguments and separates the host and the path. If no path is given, "/" is used.
2. `interact_with_server()` prints the starting message and calls the other functions in order.
3. `create_socket_and_connect()` resolves the host with getaddrinfo(), creates a TCP socket and connects to port 80.
4. `send_http_get_request()` builds the GET request and writes it to the socket.
5. `receive_http_response()` reads from the socket and prints the data until the server closes the connection.
6. The socket is closed with close().

Design decisions

- The request uses HTTP/1.0 with a Host header. With HTTP/1.0 the server closes the connection after the reply and sends the body as it is (no chunked encoding), so the end of the response is simply the point where recv() returns 0. The Host header is still needed for servers that host many websites on one IP.
- getaddrinfo() is used with AF_UNSPEC, so both hostnames and IP addresses (IPv4 or IPv6) work. If a host has more than one address, each one is tried.

- Timeouts: connect() is done in non-blocking mode and poll() waits at most 5 seconds for it. While reading, poll() waits at most 10 seconds for data. This way the client never hangs on a dead server.
- If a step fails, the function prints an error and ends the program with exit(EXIT_FAILURE), so the next function is never called with a bad socket.
- SIGPIPE is ignored, so if the server drops the connection during send() an error message is printed instead of the program being killed silently.

3. FUNCTIONS USED

Function	Input arguments	Output	Work done
`create_socket_and_connect`	`const char *hostname`	`int` (socket descriptor)	Resolves the host, creates the socket and connects to port 80
`send_http_get_request`	`int sockfd, const char *hostname, const char *path`	`void`	Sends the GET request for the given path
`receive_http_response`	`int sockfd`	`void`	Receives the reply and prints it
`interact_with_server`	`const char *hostname, const char *path`	`void`	Calls the above functions in order and closes the socket
`main`	`int argc, char *argv[]`	`int` (0 = success)	Reads command line arguments and starts the client

4. COMPILATION AND EXECUTION

```
$ gcc http_client.c -o client
$ ./client <hostname | IP address> [path]
```

The second argument is optional. A full URL such as http://example.com/index.html is also accepted; the host and path are taken from it.

5. ERROR HANDLING

Error messages go to stderr and the program exits with status 1.

Situation	Message printed
No argument / too many arguments	Usage : ./client <hostname | IP address> [path]
Hostname cannot be resolved	ERROR: cannot resolve host "<host>" (<reason>)
Server refuses or cannot be reached	ERROR: cannot connect to <host>:80 (<reason>)
No reply to connect within 5 s	ERROR: cannot connect to <host>:80 (Connection timed out)
send() fails	ERROR: send failed (<reason>)
No data from server for 10 s	ERROR: no data from server for 10 seconds
Server closes without replying	ERROR: server closed the connection without a response

Situation	Message printed
Other scheme like https:// or ftp://	ERROR: only http:// is supported
Empty or too long host / path	ERROR: invalid address "<arg>" or ERROR: path is too long

6. TEST CASES

No.	Command	What is checked	Result
1	`./client 127.0.0.1`	Server given as an IP address (local web server)	200 OK, page printed
2	`./client example.com`	Server given as a hostname	200 OK, page printed
3	`./client localhost about.html`	Hostname with a path argument	200 OK, page printed
4	`./client http://127.0.0.1/nothing.html`	URL input, object not on server	404 reply printed
5	`./client no.such.host.invalid`	Host that does not exist	Error printed
6	`./client 127.0.0.2`	No server listening on port 80	Error printed
7	`./client 10.255.255.1`	Unreachable address	Timed out after 5 s
8	`./client`	No argument given	Usage printed

7. OUTPUT OF TEST CASES

Test 1: IP address

```
$ ./client 127.0.0.1
Interacting with server 127.0.0.1...

HTTP GET request sent to 127.0.0.1

HTTP/1.0 200 OK
Server: SimpleHTTP/0.6 Python/3.14.2
Date: Mon, 19 Oct 2026 09:32:40 GMT
Content-type: text/html
Content-Length: 91
Last-Modified: Mon, 19 Oct 2026 09:32:31 GMT

<!DOCTYPE html><html><head><title>Test</title></head><body><p>It
    works!</p></body></html>


Total bytes received: 276
```

Test 2: Hostname (long HTML line shortened)

```
$ ./client example.com
Interacting with server example.com...

HTTP GET request sent to example.com

HTTP/1.1 200 OK
Date: Mon, 19 Oct 2026 09:32:43 GMT
```

```
Content-Type: text/html; charset=utf-8
Connection: close
Server: cloudflare
Last-Modified: Tue, 13 Oct 2026 20:08:28 GMT
Allow: GET, HEAD
Accept-Ranges: bytes
Age: 5629
cf-cache-status: HIT
CF-RAY: a4cec6f1db45d04b-DEL
alt-svc: h3=":443"; ma=86400

<!doctype html><html lang=en><head><meta charset=utf-8> ... <title>Example
   Domain</title>
... <p>This domain is for use in documentation examples without needing
   permission. ...
... </body></html>


Total bytes received: 891
```

Even though the request is HTTP/1.0, the server answers with "HTTP/1.1 200 OK". This is normal. A server replies with the highest version it supports, but it still follows HTTP/1.0 rules for the connection.

Test 3: Hostname with path

```
$ ./client localhost about.html
Interacting with server localhost...

HTTP GET request sent to localhost

HTTP/1.0 200 OK
Server: SimpleHTTP/0.6 Python/3.14.2
Date: Mon, 19 Oct 2026 09:32:42 GMT
Content-type: text/html
Content-Length: 82
Last-Modified: Mon, 19 Oct 2026 09:32:31 GMT

<html><body><h2>About page</h2><p>Served by local test server.</p></body></html>


Total bytes received: 267
```

Test 4: URL input, object not found (HTML shortened)

```
$ ./client http://127.0.0.1/nothing.html
Interacting with server 127.0.0.1...

HTTP GET request sent to 127.0.0.1

HTTP/1.0 404 File not found
Server: SimpleHTTP/0.6 Python/3.14.2
Date: Mon, 19 Oct 2026 09:32:42 GMT
Connection: close
Content-Type: text/html;charset=utf-8
Content-Length: 460

<!DOCTYPE HTML>
<html lang="en">
    ...
        <h1>Error response</h1>
        <p>Error code: 404</p>
        <p>Message: File not found.</p>
    ...
```

```
</html>


Total bytes received: 645
```

Test 5: Invalid hostname

```
$ ./client no.such.host.invalid
Interacting with server no.such.host.invalid...

ERROR: cannot resolve host "no.such.host.invalid" (Name or service not known)
```

Test 6: Connection refused

```
$ ./client 127.0.0.2
Interacting with server 127.0.0.2...

ERROR: cannot connect to 127.0.0.2:80 (Connection refused)
```

Test 7: Unreachable address

```
$ ./client 10.255.255.1
Interacting with server 10.255.255.1...

ERROR: cannot connect to 10.255.255.1:80 (Connection timed out)
```

Test 8: No argument

```
$ ./client
Usage  : ./client <hostname | IP address> [path]
Example: ./client 127.0.0.53
```

8. CONCLUSION

The client connects to any HTTP server given by hostname or IP address, sends a GET request and prints the server's reply. Through this assignment I learnt how name resolution, socket creation, connect, send and recv fit together, and how to deal with failures such as unknown hosts, refused connections and servers that do not answer.

APPENDIX: SOURCE CODE

File: `http_client.c`

```
/*
 * ------------------------------------------------------------------
 *  Network Programming and Client Server Programming
 *  Assignment : Simple HTTP Client using sockets
 *
 *  Student Name : UJJWAL JAIN
 *  Student ID   : 2024EB02528
 *
 *  Build : gcc http_client.c -o client
 *  Usage : ./client <hostname | IP address> [path]
 *          e.g. ./client 127.0.0.53
 *               ./client neverssl.com /index.html
 * ------------------------------------------------------------------
 */

#define _POSIX_C_SOURCE 200809L

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <signal.h>
#include <fcntl.h>
#include <poll.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>

#define SERVER_PORT        "80"
#define REQ_SIZE           2048
#define RECV_CHUNK         1024
#define CONNECT_TIMEOUT_MS 5000
#define READ_TIMEOUT_MS    10000

/*
 * create_socket_and_connect()
 * Input  : hostname - server name or IP address
 * Output : descriptor of a connected TCP socket
 * Resolves the host, creates a socket and connects to port 80.
 * Prints an error and terminates the program if this fails.
 */
int create_socket_and_connect(const char *hostname)
{
    struct addrinfo hints;
    struct addrinfo *list, *ai;
    int fd = -1;
    int rc, flags, last_err = 0;

    memset(&hints, 0, sizeof hints);
    hints.ai_family   = AF_UNSPEC;   /* IPv4 or IPv6, whatever the host has */
    hints.ai_socktype = SOCK_STREAM;

    rc = getaddrinfo(hostname, SERVER_PORT, &hints, &list);
    if (rc != 0) {
        fprintf(stderr, "ERROR: cannot resolve host \"%s\" (%s)\n",
                hostname, gai_strerror(rc));
        exit(EXIT_FAILURE);
    }

    for (ai = list; ai != NULL; ai = ai->ai_next) {
        fd = socket(ai->ai_family, ai->ai_socktype, ai->ai_protocol);
        if (fd == -1) {
            last_err = errno;
            continue;
        }
```

```
        /* connect in non-blocking mode so we can stop waiting
           after CONNECT_TIMEOUT_MS instead of the system default */
        flags = fcntl(fd, F_GETFL, 0);
        fcntl(fd, F_SETFL, flags | O_NONBLOCK);

        rc = connect(fd, ai->ai_addr, ai->ai_addrlen);
        if (rc == -1 && errno == EINPROGRESS) {
            struct pollfd pfd;
            int so_error = 0;
            socklen_t len = sizeof so_error;

            pfd.fd = fd;
            pfd.events = POLLOUT;
            pfd.revents = 0;

            rc = poll(&pfd, 1, CONNECT_TIMEOUT_MS);
            if (rc == 0) {
                errno = ETIMEDOUT;
                rc = -1;
            } else if (rc > 0) {
                /* socket is writable, check if the connect worked */
                getsockopt(fd, SOL_SOCKET, SO_ERROR, &so_error, &len);
                if (so_error != 0) {
                    errno = so_error;
                    rc = -1;
                } else {
                    rc = 0;
                }
            }
        }

        if (rc == 0) {
            fcntl(fd, F_SETFL, flags);   /* back to blocking mode */
            break;
        }

        last_err = errno;
        close(fd);
        fd = -1;
    }
    freeaddrinfo(list);

    if (fd == -1) {
        fprintf(stderr, "ERROR: cannot connect to %s:%s (%s)\n",
                hostname, SERVER_PORT, strerror(last_err));
        exit(EXIT_FAILURE);
    }

    return fd;
}

/*
 * send_http_get_request()
 * Input  : sockfd   - connected socket
 *          hostname - goes in the Host header
 *          path     - object to fetch, e.g. "/" or "/index.html"
 * Output : none, the GET request is written to the socket
 */
void send_http_get_request(int sockfd, const char *hostname, const char *path)
{
    char req[REQ_SIZE];
    const char *p;
    size_t left;
    ssize_t n;
    int len;

    /* HTTP/1.0: server closes the connection after the reply and does
       not use chunked encoding. Host is still sent for virtual hosts. */
    len = snprintf(req, sizeof req,
                   "GET %s HTTP/1.0\r\n"
                   "Host: %s\r\n"
                   "User-Agent: simple-http-client\r\n"
```

```
                "\r\n",
                path, hostname);
    if (len < 0 || (size_t)len >= sizeof req) {
        fprintf(stderr, "ERROR: request too large\n");
        close(sockfd);
        exit(EXIT_FAILURE);
    }

    p = req;
    left = (size_t)len;
    while (left > 0) {
        n = send(sockfd, p, left, 0);
        if (n == -1) {
            if (errno == EINTR)
                continue;
            fprintf(stderr, "ERROR: send failed (%s)\n", strerror(errno));
            close(sockfd);
            exit(EXIT_FAILURE);
        }
        p += n;
        left -= (size_t)n;
    }

    printf("HTTP GET request sent to %s\n\n", hostname);
    fflush(stdout);
}

/*
 * receive_http_response()
 * Input  : sockfd - connected socket
 * Output : none, the response is printed on stdout
 * Reads until the server closes the connection. poll() is used so the
 * client gives up if the server goes quiet for READ_TIMEOUT_MS.
 */
void receive_http_response(int sockfd)
{
    char chunk[RECV_CHUNK];
    struct pollfd pfd;
    size_t received = 0;
    ssize_t n;
    int rc, closed = 0;

    pfd.fd = sockfd;
    pfd.events = POLLIN;
    pfd.revents = 0;

    for (;;) {
        rc = poll(&pfd, 1, READ_TIMEOUT_MS);
        if (rc == 0) {
            fprintf(stderr, "\nERROR: no data from server for %d seconds\n",
                    READ_TIMEOUT_MS / 1000);
            break;
        }
        if (rc == -1) {
            if (errno == EINTR)
                continue;
            fprintf(stderr, "\nERROR: poll failed (%s)\n", strerror(errno));
            break;
        }

        n = recv(sockfd, chunk, sizeof chunk, 0);
        if (n == 0) {
            closed = 1;   /* server closed the connection, reply is complete */
            break;
        }
        if (n == -1) {
            if (errno == EINTR)
                continue;
            fprintf(stderr, "\nERROR: recv failed (%s)\n", strerror(errno));
            break;
        }
```

```
        fwrite(chunk, 1, (size_t)n, stdout);
        received += (size_t)n;
    }

    if (received > 0)
        printf("\n\nTotal bytes received: %zu\n", received);
    else if (closed)
        fprintf(stderr, "ERROR: server closed the connection without a response\n");
    fflush(stdout);
}

/*
 * interact_with_server()
 * Input  : hostname, path
 * Output : none
 * Full exchange: connect -> send GET -> print response -> close.
 */
void interact_with_server(const char *hostname, const char *path)
{
    printf("Interacting with server %s...\n\n", hostname);
    fflush(stdout);

    // Call function to create a socket and connect to the server
    int sockfd = create_socket_and_connect(hostname);

    // Call function to send an HTTP GET request
    send_http_get_request(sockfd, hostname, path);

    // Call function to receive and print the HTTP response
    receive_http_response(sockfd);

    // Call function to close the socket
    close(sockfd);
}

/*
 * main()
 * Input  : argv[1] = hostname or IP (an http:// URL also works)
 *          argv[2] = path (optional, default "/")
 * Output : 0 on success, 1 on bad input
 */
int main(int argc, char *argv[])
{
    char hostname[256];
    char path[1024] = "/";
    const char *target;
    size_t hostlen;
    int rc;

    if (argc != 2 && argc != 3) {
        fprintf(stderr, "Usage  : %s <hostname | IP address> [path]\n", argv[0]);
        fprintf(stderr, "Example: %s 127.0.0.53\n", argv[0]);
        return EXIT_FAILURE;
    }

    /* if the server drops the connection while we are sending,
       report the error instead of being killed by SIGPIPE */
    signal(SIGPIPE, SIG_IGN);

    target = argv[1];
    if (strncmp(target, "http://", 7) == 0) {
        target += 7;
    } else if (strstr(target, "://") != NULL) {
        fprintf(stderr, "ERROR: only http:// is supported\n");
        return EXIT_FAILURE;
    }

    /* split "host/some/path" into host and path */
    hostlen = strcspn(target, "/");
    if (hostlen == 0 || hostlen >= sizeof hostname ||
        strlen(target + hostlen) >= sizeof path) {
        fprintf(stderr, "ERROR: invalid address \"%s\"\n", argv[1]);
```

```
        return EXIT_FAILURE;
    }
    sscanf(target, "%255[^/]%1023s", hostname, path);

    if (argc == 3) {
        rc = snprintf(path, sizeof path, "%s%s",
                      argv[2][0] == '/' ? "" : "/", argv[2]);
        if (rc < 0 || (size_t)rc >= sizeof path) {
            fprintf(stderr, "ERROR: path is too long\n");
            return EXIT_FAILURE;
        }
    }

    // Call fuction to interact with the server
    interact_with_server(hostname, path);

    return 0;
}
```